C++ COMPLETE REFERENCE LIBRARY • MODULE 15

15 VIRTUAL FUNCTIONS AND ABSTRACTION — Complete Theory & Study Book

Comprehensive Offline Theory, Memory Diagrams, Classroom C++ Programs, Practice Questions & Solutions

Module Table of Contents

1. **Module Overview & Study Guide** (README)
2. **Theory Chapter:** 01 virtual functions vtable and vptr
3. **Theory Chapter:** 02 pure virtual functions and abstract classes
4. **Theory Chapter:** 03 virtual destructors override and final
5. **Visual Memory & Concept Diagrams** (2 Vector Diagrams)
6. **Annotated C++ Source Programs** (4 Programs)
7. **Practice Question Bank:** conceptual questions
8. **Practice Question Bank:** debugging questions
9. **Practice Question Bank:** predict the output
10. **Practice Question Bank:** programming questions
11. **Step-by-Step Solutions:** conceptual solutions
12. **Step-by-Step Solutions:** output solutions
13. **Step-by-Step Solutions:** programming solutions

15 — Virtual Functions and Abstraction

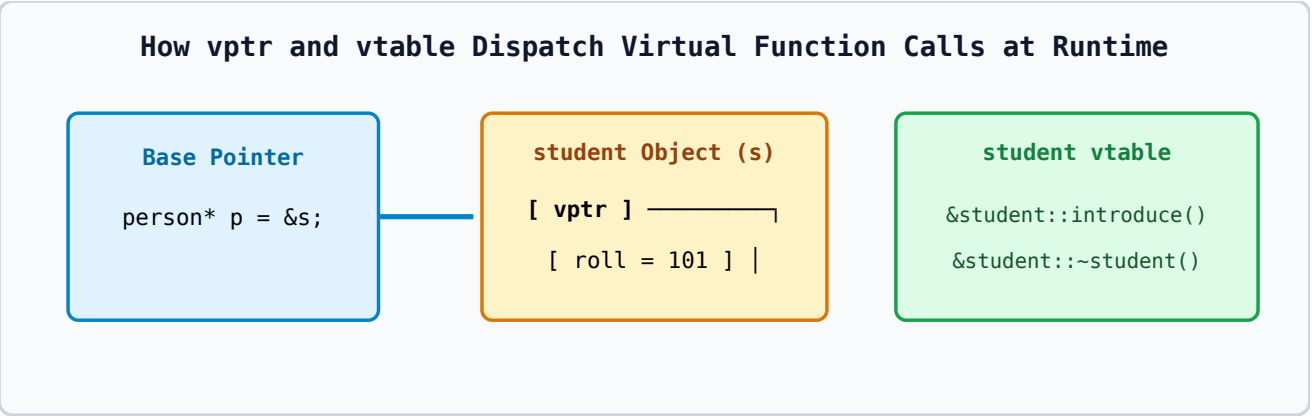
This module explains `virtual` functions, the `vtable` and `vptr` dispatch mechanism, pure virtual functions (`virtual void f() = 0;`), abstract base classes, `override` and `final`, and why polymorphic base classes require a `virtual` destructor.

Directory Contents

- **Theory (`01_THEORY/`)**:
  - `01_virtual_functions_vtable_and_vptr.md` (./01_THEORY/01_virtual_functions_vtable_and_vptr.md)
  - `02_pure_virtual_functions_and_abstract_classes.md` (./01_THEORY/02_pure_virtual_functions_and_abstract_classes.md)
  - `03_virtual_destructors_override_and_final.md` (./01_THEORY/03_virtual_destructors_override_and_final.md)
- **Programs (`02_PROGRAMS/`)**:

  - `01_virtual_function_dispatch.cpp` (./02_PROGRAMS/01_virtual_function_dispatch.cpp) (Category A)
  - `02_abstract_class_and_pure_virtual.cpp` (./02_PROGRAMS/02_abstract_class_and_pure_virtual.cpp) (Category A)
  - `03_abstract_instantiation_compile_error.cpp` (./02_PROGRAMS/03_abstract_instantiation_compile_error.cpp) (Category B)
  - `04_virtual_destructor_importance.cpp` (./02_PROGRAMS/04_virtual_destructor_importance.cpp) (Category A)
- **Practice Questions (`03_PRACTICE_QUESTIONS/`)** & **Detailed Solutions (`04_DETAILED_SOLUTIONS/`)**
- **Diagrams (`05_DIAGRAMS/`)**:
  - `diagram_01_vtable_and_vptr_dispatch.svg` (./05_DIAGRAMS/diagram_01_vtable_and_vptr_dispatch.svg)
  - `diagram_02_abstract_class_hierarchy.svg` (./05_DIAGRAMS/diagram_02_abstract_class_hierarchy.svg)

01 — How Virtual Functions Work Internally: `vtable` and `vptr`



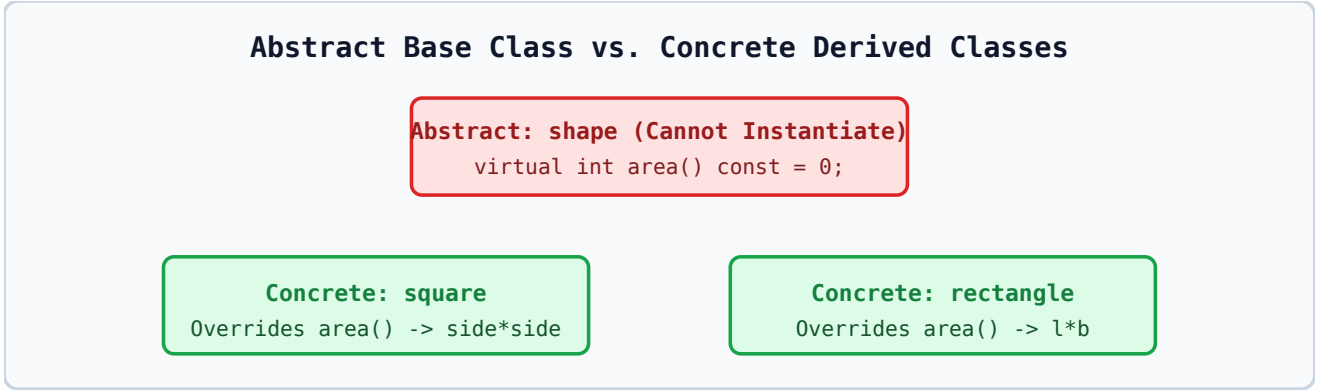
vtable and vptr Dispatch

1. The `vtable` and `vptr` Mechanism

When a class declares or inherits at least one `virtual` function:

1. **Per-Class Virtual Table (`vtable`)**: The compiler creates one static table of function pointers for that class containing the addresses of the most-derived `virtual` functions for that class.
2. **Per-Object Virtual Pointer (`vptr`)**: The compiler inserts a hidden pointer (`vptr`, 8 bytes on 64-bit systems) inside every object of that class, initialized by the constructor to point to that class's `vtable`.
3. **Runtime Lookup (`ptr→show()`)**: Follows `ptr→vptr` to the `vtable` and invokes the function pointer stored in the corresponding slot!

02 — Pure Virtual Functions (`= 0`) and Abstract Classes



Abstract Class Hierarchy

1. Definitions

- **Pure Virtual Function**: A virtual function declared with `= 0` (`virtual void area() const = 0;`).
- **Abstract Class**: Any class containing **at least one** pure virtual function.

2. Strict Rules of Abstract Classes (`[class.abstract]`)

1. **Cannot Be Instantiated Directly**: Declaring `shape s;` is a **Compile-Time Error**.
2. **Pointers and References Are Allowed**: You **can** declare `shape* ptr;` and `shape& ref = circle_obj;` .
3. **Derived Class Rule**: If a derived class does **not** override all pure virtual functions of its base class, the derived class **also becomes abstract** and cannot be instantiated!

03 — Virtual Destructors, `override`, and `final`

1. Why Base Classes Must Have a `virtual` Destructor

If you delete a derived object through a base class pointer (`base* p = new derived(); delete p;`) and `~base()` is **not** `virtual`, the C++ standard (`[expr.delete]`) states that **the behavior is Undefined Behavior** (typically only `~base()` runs and `~derived()` is never called, leaking derived resources!).

- **Rule**: Always declare `virtual ~base() {}` (or `virtual ~base() = default;`) in any polymorphic base class!
- **Can a Constructor be `virtual`?**: **NO!** A constructor cannot be `virtual` because the object's `vptr` is not yet initialized until the constructor runs.

Visual Concept & Memory Diagrams (15 VIRTUAL FUNCTIONS AND ABSTRACTION)

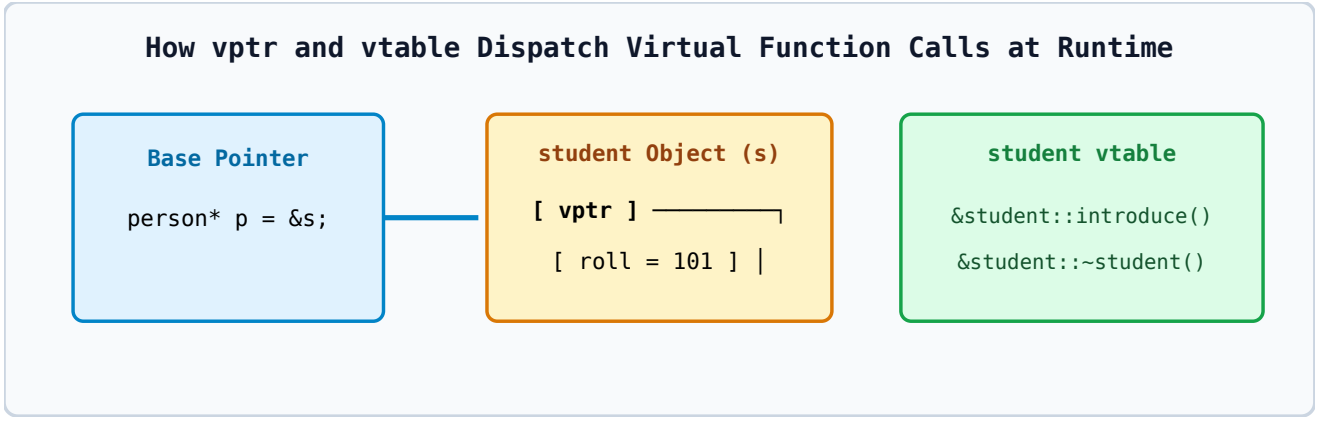


Diagram: diagram 01 vtable and vptr dispatch

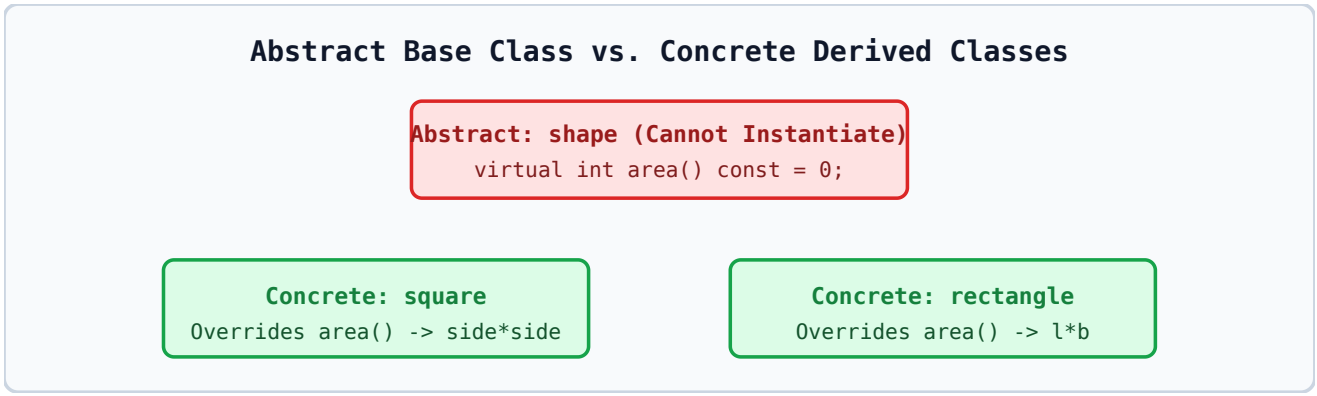


Diagram: diagram 02 abstract class hierarchy

Annotated C++ Programs (15 VIRTUAL FUNCTIONS AND

ABSTRACTION)

Program: `01_virtual_function_dispatch.cpp`

C++ SOURCE FILE • 01_virtual_function_dispatch.cpp

```cpp
// Program Title: Runtime Polymorphism Using Virtual Functions
// Topic: Virtual Functions and Abstraction
// Purpose: Demonstrate dynamic dispatch via base pointer pointing to different derived
objects.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/01_virtual_functions_vtable_and_vptr.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

class person {
public:
    virtual void introduce() const {
        cout << "I am a person." << endl;
    }
    virtual ~person() = default;
};

class student : public person {
public:
    void introduce() const override {
        cout << "I am a student." << endl;
    }
};

class teacher : public person {
public:
    void introduce() const override {
        cout << "I am a teacher." << endl;
    }
};

int main() {
    student s;
    teacher t;

    person* p = &s;
    p->introduce();

    p = &t;
    p->introduce();

    return 0;
}

/*
Expected Deterministic Output:
I am a student.
```

```
I am a teacher.
*/
```

Program: `02_abstract_class_and_pure_virtual.cpp`

C++ SOURCE FILE • 02_abstract_class_and_pure_virtual.cpp

```cpp
// Program Title: Abstract Base Class with Pure Virtual Function (= 0)
// Topic: Virtual Functions and Abstraction — Abstract Classes
// Purpose: Demonstrate an abstract class 'shape' and concrete derived classes 'square' and
'rectangle'.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/02_pure_virtual_functions_and_abstract_classes.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

class shape {
public:
    virtual int area() const = 0; // Pure virtual function
    virtual ~shape() = default;
};

class square : public shape {
private:
    int side;

public:
    square(int s) : side(s) {}

    int area() const override {
        return side * side;
    }
};

int main() {
    square sq(6);
    shape* ptr = &sq;
    cout << "Square area via abstract pointer: " << ptr->area() << endl;
    return 0;
}

/*
Expected Deterministic Output:
Square area via abstract pointer: 36
*/
```

Program: `03_abstract_instantiation_compile_error.cpp`

C++ SOURCE FILE • 03_abstract_instantiation_compile_error.cpp

```cpp
// INTENTIONAL COMPILE-TIME ERROR DEMONSTRATION
// Program Title: Attempting to Instantiate an Abstract Class Directly
// Topic: Virtual Functions and Abstraction — Abstract Class Rules
// Purpose: Show why creating an object of a class containing a pure virtual function fails
at compile time.
// Target C++ Standard: C++23
// Related Theory File: ../01_THEORY/02_pure_virtual_functions_and_abstract_classes.md
// Program Classification: Category B — Intentional compilation-error demonstration

#include <iostream>
using namespace std;

class shape {
public:
    virtual void draw() const = 0;
};

int main() {
    shape s; // COMPILE ERROR: cannot declare variable 's' to be of abstract type 'shape'
    return 0;
}

/*
Why Compilation Fails:
'shape' has a pure virtual function 'virtual void draw() const = 0;', making 'shape' an
abstract class ([class.abstract]).
Abstract classes cannot be instantiated directly; instantiate a concrete derived class that
overrides 'draw()'.
*/
```

Program: `04_virtual_destructor_importance.cpp`

C++ SOURCE FILE • 04_virtual_destructor_importance.cpp

```cpp
// Program Title: Virtual Destructor Ensuring Complete Derived Cleanup via Base Pointer
// Topic: Virtual Functions and Abstraction — Virtual Destructors
// Purpose: Show how 'virtual ~super()' guarantees that '~sub()' executes when deleting a
'sub' object via 'super*'.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/03_virtual_destructors_override_and_final.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

class super {
public:
    super() {
        cout << "1. super constructed" << endl;
    }
    virtual ~super() {
        cout << "4. super destroyed" << endl;
    }
};

class sub : public super {
public:
    sub() {
        cout << "2. sub constructed" << endl;
    }
    ~sub() override {
        cout << "3. sub destroyed" << endl;
    }
};

int main() {
    super* ptr = new sub();
    delete ptr; // Safely calls ~sub() first, then ~super()!
    return 0;
}

/*
Expected Deterministic Output:
1. super constructed
2. sub constructed
3. sub destroyed
4. super destroyed
*/
```

Conceptual Questions — Module 15: Virtual Functions and Abstraction

Q1. Why can't a constructor be `virtual` in C++, while a destructor in a base class **should** be `virtual`?
Q2. What is a pure virtual function, and can a pure virtual function have a function body defined outside the class?

Debugging Questions — Module 15

Q4. Why does `derived d;` fail to compile if `base` has two pure virtual functions (`f1() = 0` and `f2() = 0`) and `derived` only overrides `f1()`?

Predict the Output Questions — Module 15

Q3. What happens to `sizeof(MyClass)` on a 64-bit platform when you change one member function from non-virtual to `virtual`?

Programming Questions — Module 15

Q5. Write a C++23 classroom program with an abstract class `account` (`virtual void calculate_interest() = 0;`) and derived classes `savings_account` and `current_account`.

Detailed Solutions — Conceptual Questions (Module 15)

- **Q1 Solution**: A constructor initializes the object and sets up its `vptr` to point to the class's `vtable`; before construction begins, no `vptr` exists to dispatch virtually. A base destructor must be `virtual` so `delete base_ptr;` invokes the derived destructor first instead of causing Undefined Behavior.
- **Q2 Solution**: A pure virtual function is declared with `= 0` and makes its class abstract. Yes, C++ allows a pure virtual function to have a body defined outside the class (which derived classes can call statically via `Base::func()`), though the class remains abstract! Pure virtual destructors (`virtual ~Base() = 0;`) **must** provide a body.

Detailed Solutions — Predict the Output & Debugging (Module 15)

- **Q3 Solution**: Adding the first `virtual` function adds one hidden `vptr` (8 bytes on a 64-bit system, plus any alignment padding) to every object of `MyClass`. Adding a second or third `virtual` function adds entries to the class's static `vtable` without adding any more pointers to the object itself!
- **Q4 Solution**: Any derived class that leaves even one inherited pure virtual function un-overridden remains an **abstract class** itself and cannot be instantiated.

Detailed Solutions — Programming Questions (Module 15)

CPP

```cpp
#include <iostream>
using namespace std;

class account {
public:
    virtual void calculate_interest() const = 0;
    virtual ~account() = default;
};

class savings_account : public account {
    int balance;
public:
    savings_account(int b) : balance(b) {}
    void calculate_interest() const override {
        cout << "Savings Interest: " << (balance * 4 / 100) << endl;
    }
};

class current_account : public account {
public:
    void calculate_interest() const override {
        cout << "Current Interest: 0" << endl;
    }
};

int main() {
    savings_account sa(1000);
    current_account ca;
    account* p = &sa;
    p->calculate_interest();
    p = &ca;
    p->calculate_interest();
    return 0;
}
```